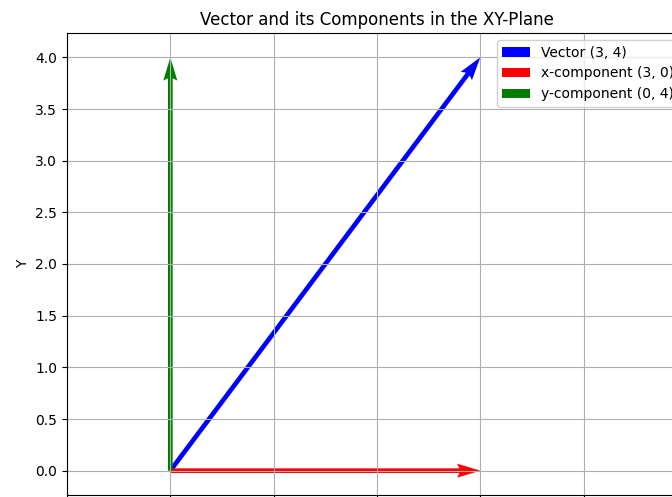


Vectors

- A vector is generally a 1-dimensional array of numbers $v = [x_1, x_2, \dots, x_n]$, where the numbers are called **components**
- A vector with two components has a geometrical interpretation in plane
- The vector $v = [3, 4]$ means 3 steps to the direction of positive x -axis and 4 steps to the direction of positive y axis from the start
- Negative values would mean moves to the opposite direction



Vector algebra

- Vectors are one-dimensional matrices
- So either row or column
- Because they are matrices, the rules of matrix algebra are applied

Example

Let $a = [1, 3, 6]$, $b = [2, 5, 7]$ and $c = \begin{bmatrix} 9 \\ 10 \\ 11 \end{bmatrix}$

1. $a + b = [1 + 2, 3 + 5, 6 + 7] = [3, 8, 13]$

2. $3a = 3[1, 3, 6] = [3 \cdot 1, 3 \cdot 3, 3 \cdot 6] = [3, 9, 18]$

3. $0a = [0, 0, 0]$

4. $a + c$ or $b + c$ do not exist. (Dimensions do not match)

5. $ac = [1, 3, 6] \begin{bmatrix} 9 \\ 10 \\ 11 \end{bmatrix} = [1 \cdot 9 + 3 \cdot 10 + 6 \cdot 11] = [105] = 105$

$$(a = [1, 3, 6], b = [2, 5, 7] \text{ and } c = \begin{bmatrix} 9 \\ 10 \\ 11 \end{bmatrix})$$

6. $ab = [1, 3, 6][2, 5, 7]$ does not exist. Rows are multiplied by columns, but the columns have only one element.

Vector norms

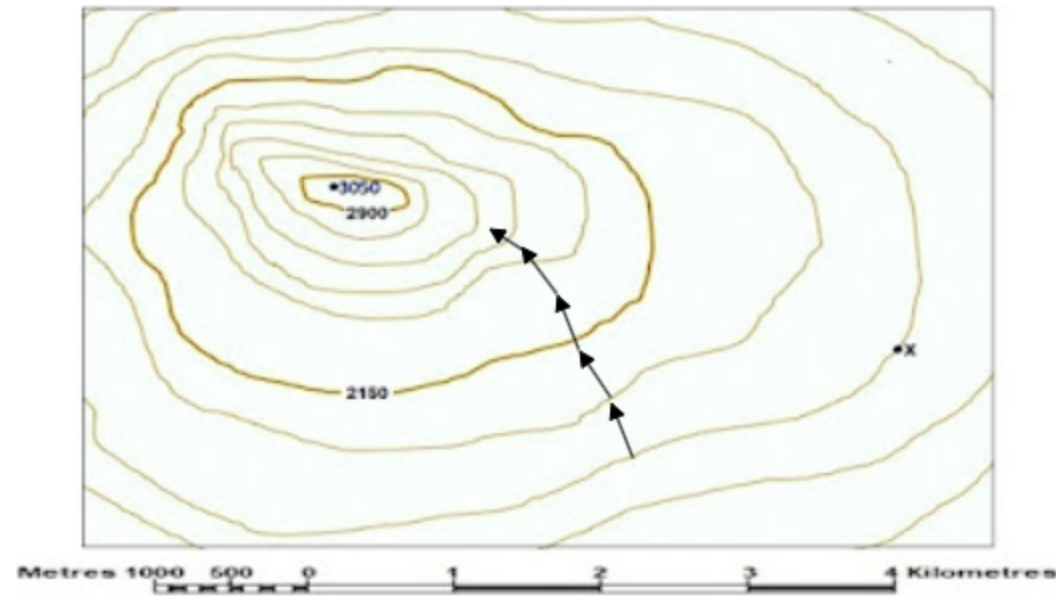
- Norm is a measure of magnitude
- In two or three dimensional cases of an important special case of the norm is the length of a vector
- Several definitions for norm exist
- Let $x = [1, -2, 3]$
- Euclidian norm: $\|x\|_2 = \sqrt{1^2 + (-2)^2 + 3^2} = \sqrt{14}$ (The geometric length)
- **Manhattan norm**: $\|x\|_1 = |1| + |-2| + |3| = 1 + 2 + 3 = 6$

The most important properties of a norm are non-negativity, $\|x\| \geq 0$, and $\|x\| = 0 \iff x = [0, 0, \dots, 0]$ (or a column vector of all zeros)

Gradient vector

What is the Gradient?

Imagine you're hiking in the mountains under low visibility conditions, and you want to find to the top. One strategy is to gain as much elevation as soon as possible. So you take the steepest possible route and continue that direction for some distance, and then update the direction.



- In math terms, the **gradient** is a **vector** (a list of numbers) that points in the **direction of the steepest increase of a function**.
- If you have a function with multiple inputs (like $z = f(x, y)$), the gradient tells you the relative change in the function value as you tweak each input x and y .
 - relative means here the change in z divided by the change in x and y
- If the gradient of the function has **all components equal to zero**, it is **possible** that we have found a minimum or maximum point of the function
 - $\nabla f(a, b) = 0 \longrightarrow$ flat point at (a, b) (either the top of a hill or the bottom of a valley)

Gradients in data science

In data science, gradients are utilized in algorithmic level. Here's why they matter:

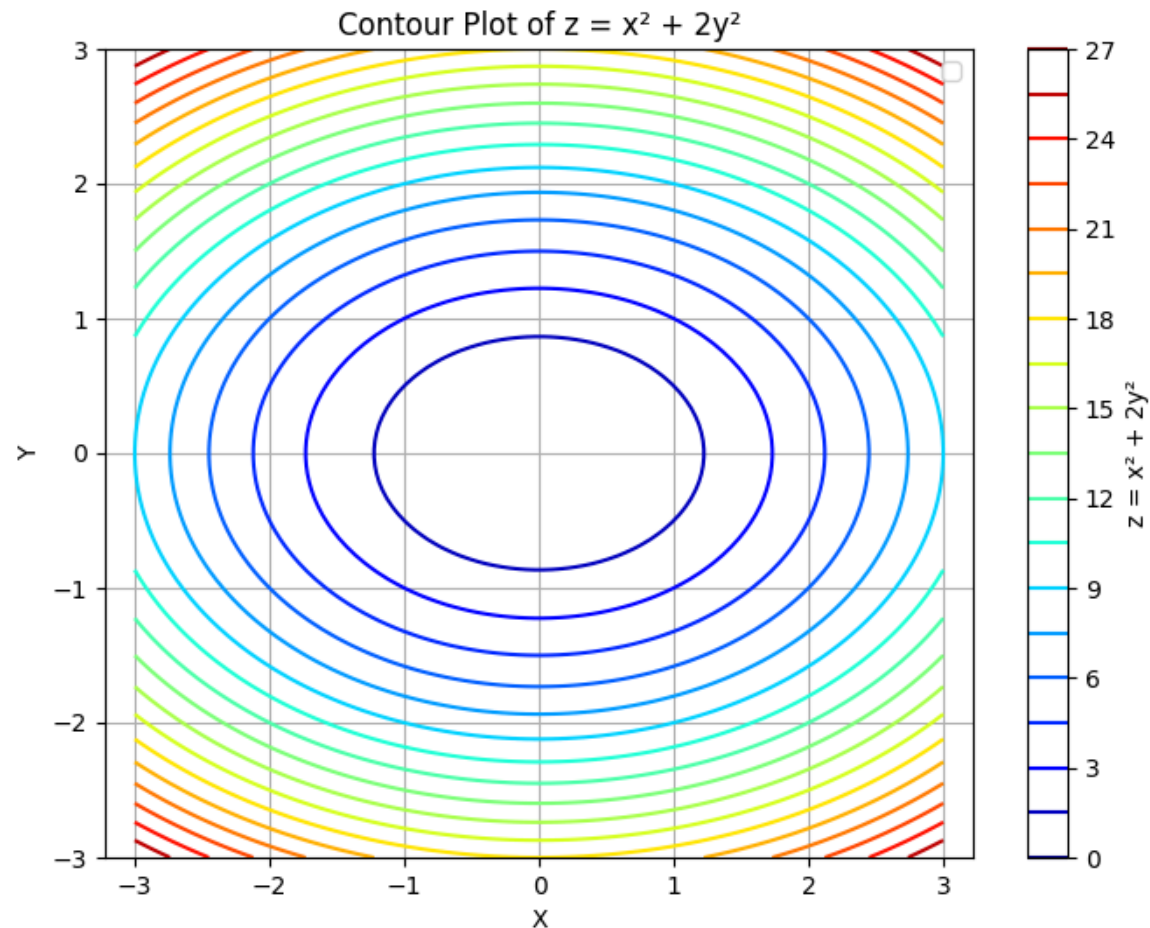
1. **Optimization:** Many machine learning algorithms (like linear regression, neural networks, etc.) rely on finding the "best" values for parameters. The gradient helps us figure out how to adjust these parameters to improve performance.
2. **Understanding Relationships:** Gradients can tell you how sensitive a function is to changes in its inputs. For example, in a model predicting house prices, the gradient can show how much the price changes as you increase the size of the house or the number of bedrooms.
3. **Finding Minimums:** In machine learning, we often want to minimize a "loss function" (a measure of how wrong our predictions are). The gradient guides us downhill to find the lowest point, where the model performs best.

Plots of functions in two independent variables

- Let $z = f(x, y) = x^2 + 2y^2$ the variables x and y are called **independent** variables and z is called **dependent** variable
- or x and y are **inputs** and z the **output**
 - you are free to change x and y , but not z
- The best way to plot a function of two variables is to use the **contour** plot
- contour plot is like a map where the value of z is show as a closed curve and x and y variables are the coordinate axes

Example $z = x^2 + 2y^2$

- Along each curve the value of z is constant



Numerical approximation of the gradient

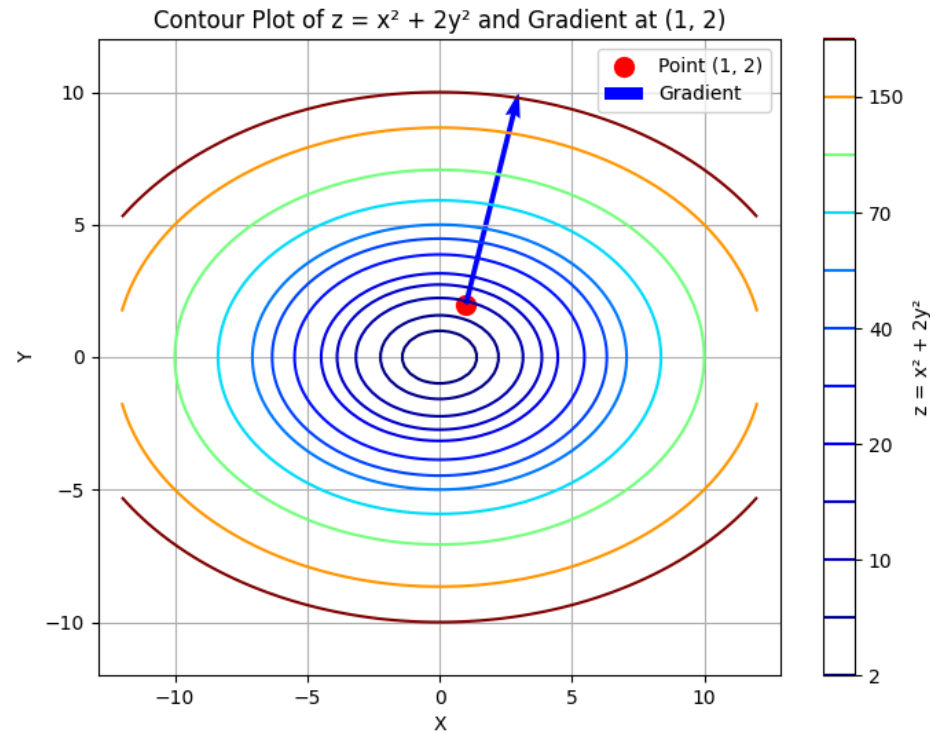
- Let us now calculate a numerical approximation of the gradient of $f(x, y) = x^2 + 2y^2$ at $(x, y) = (1, 2)$

```
def fun(x,y):  
    return x**2 + 2*y**2  
  
x, y = 1,2 #The points  
  
Del_x, Del_y = 0.001, 0.001 #Changes in x and y  
  
Del_f_Del_x = (fun(x + Del_x,y) - fun(x,y))/Del_x #Change in x and y kept at 2  
Del_f_Del_y = (fun(x,y + Del_y) - fun(x,y))/Del_y #Change in y and x kept at 1  
  
print(f"The gradient of f(x) = x**2 + y**2 at (x,y) = ({x},{y}) is {[Del_f_Del_x,Del_f_Del_y]}")
```

```
The gradient of f(x) = x**2 + y**2 at (x,y) = (1,2) is [2.00099999999999195, 8.0019999999999398]
```

So, at the point $(1, 2)$, the gradient is $\nabla f(1, 2) = (2, 8)$. This means:

- The steepest increase in f is in the direction of the vector $(2, 8)$.
- The slope in that direction is $\sqrt{2^2 + 8^2} = \sqrt{68}$.



Negative gradient

- In data science reducing the function value is often important
- The direction of steepest **descent** of the function is simply the negative gradient $-\nabla f(x)$

Gradients in Data Science: A Simple Example

Let's say you're building a linear regression model to predict house prices. Your model might look like this:

$$\text{Price} = w_1 \cdot \text{Size} + w_2 \cdot \text{Bedrooms} + b$$

Here, w_1 , w_2 , and b are parameters you need to tune.

To find the best values for w_1 , w_2 , and b , you define a **loss function** (e.g., mean squared error) that measures how far off your predictions are from the actual prices. The gradient of this loss function tells you:

- How to adjust w_1 , w_2 , and b to make your predictions better.
- How to make the loss smaller: move to the direction of **negative gradient**

This process is called **gradient descent**, and it's one of the most important algorithms in machine learning

Example: Calculating the Gradient of a Function

- In the previous example an approximate gradient of function was calculated by making tiny changes in the variables
- `numdifftools` gives a much better approximation in the order of machine precision
- Let's again study the function $f(x, y) = x^2 + 2y^2$
- We want to compute its gradient at the point $(x, y) = (1, 2)$.

Step 1: Install `numdifftools`

If you don't already have `numdifftools` installed, you can install it using pip:

```
pip install numdifftools
```

Step 2: Write the Code

- The idea is simply to write a function and * `numdifftools` then takes the care of the rest

```
import numdifftools as nd

# Define the function
def f(x):
    return x[0]**2 + x[1]**2 # f(x, y) = x^2 + y^2

# Create a gradient object
gradient = nd.Gradient(f)

# Evaluate the gradient at the point (x, y) = (1, 2)
point = [1, 2]
gradient_at_point = gradient(point)

print("Gradient at (1, 2):", gradient_at_point)
```

Step 3: Run the Code

When you run the code, you'll get the following output:

```
Gradient at (1, 2): [2. 4.]
```

- The result is same that we would get with analytical methods

Steepest descent

- Often the goal is to find the smallest value of a function
- For example, in machine learning minimization of the loss function leads to a model that can better describe the data in the learning phase
- Steepest descent algorithm uses the negative gradient $-\nabla f$ as the direction to improve the model parameters

- If we start at the point (x_0, y_0) the gradient is evaluated the point, $\nabla f(x_0, y_0)$ the a step of length γ is taken in to the direction of the **negative** gradient and a new point $(x_1, y_1) = (x_0, y_0) - \gamma \nabla(f(x_0, y_0))$ is obtained
- If the value of step length γ is too small the convergence to the optimum would take long, if it is too large it can overshoot the optimum and start to oscillate and never get to the opitimum
- One way to determine a suitable γ is [backtracking](#)

Example

Let $f(x, y) = x^2 + 2y^2$. Starting from the point $(x, y) = (1, 2)$ the following script illustrates the evaluation of steepest descent method.

```
import numpy as np

def fun(x):
    return np.array(x[0]**2 + 2*x[1]**2)

def grad(x):
    return np.array([2*x[0], 4*x[1]])
```

```
x = np.array([1,2]) #Initial point
counter = 0 #iter counter
max_iter = 500 #max nr of iterations
gamma = 0.01 #Step lenght
toler = 1e-4
#flag = 0 grad less than tol, flag = 1 max_iter reached
while True:
    counter += 1
    x = x - gamma*grad(x)
    if np.linalg.norm(grad(x)) < toler:
        flag = 0
        break
    elif counter >= max_iter:
        flag = 1
        break

print("flag = ", flag, "x =", x, "iterations =", counter, "gradient =", grad(x))
```

Steepest descent with numdifftools

- Assume that we want to fit the model $y = ax^2 + bx + c$ to data
- So we have the same number of observations for x and y variables
- This problem can be solved using the least squares method.
- In practise this would be solved using regression fitting of e.g. `numpy`, but let us illustrate how to solve this with the gradient method


```

from numpy.random import normal, randint
from numpy import zeros, array
from numpy.linalg import norm
import numdifftools as nd
# DATA GENERATION PART
def fun(x):
    #data generation for fit
    return 0.02*x**2 + 2*x + 2 + normal(loc = 0, scale = 0.1)

#Generate data
xdata = randint(low = 1, high = 50, size = (10,1))
ydata = zeros((10,1))

for i in range(10):
    ydata[i] = fun(xdata[i])

```

```

#OPTIMIZATION PART
def loss(par):
    return norm(ydata - (par[0]*xdata**2 + par[1]*xdata + par[2]))

#Find the gradient
grad = nd.Gradient(loss)

x = array([0.02,2,2]) #Initial point
counter = 0 #iter counter
max_iter = 150 #max nr of iterations
gamma = 0.01 #Step lenght
toler = 1e-3

#flag = 0 grad less than tol, flag = 1 max_iter reached
while True:
    counter += 1
    x = x - gamma*grad(x)
    if norm(grad(x)) < toler:
        flag = 0
        break
    elif counter >= max_iter:
        flag = 1
        break

if flag == 0:
    print("Gradient converged to 0 within tolerance.")
else:
    print("Gradient non-zero")

print("flag = ", flag, "\nx = ", x, "\niterations = ", counter, "\ngradient = ", grad(x))

```

Gradient converged to 0 within tolerance

flag = 0

x = [0.02 2.000006 1.94866337]

iterations = 24

gradient = [2.26544532e-08 5.83935733e-06 9.38204429e-04]

Why is the gradient method used in Data Science

- It is common knowledge that the steepest descent method is a poor performer in general function minimization
- There are couple of reasons why it is used in data sciences
 1. Model overfitting is a major problem. Steepest descent tends to stop early and does not "fit too well" the model
 2. More sophisticated function minimization methods try to estimate second derivative information (like curvature of surface) instead of just the slope. However, this information needs a matrix and its updates and they are too expensive (time consuming and storage)
 3. For example a gradient vector with 10^6 elements can be stored and updated but a matrix with 10^6 rows and 10^6 columns is hard

